Recommender Systems

Course Code: MEAD-753

Lab Practical File

Submitted to: Dr. Kriti Saroha

Submitted by

Shantanu Shukla

01211805424

in partial fulfillment for the award of the degree

of

Master of Technology (AI & DS)

batch of 2024 - 26 (3rd Semester)

Center for Development of Advanced Computing, Noida

Affiliated to Guru Gobind Singh Indraprastha University

INDEX

S. No.	Title	Page No.	Signature
1	User-User Collaborative Filtering using Pearson Similarity Index in Excel	3	
2	Item-Item Collaborative Filtering using Cosine Similarity Index in Excel	4	
3	User-User Collaborative Filtering in Python	5	
4	Item-Item Collaborative Filtering in Python	12	
5	Content Based Filtering in Excel	17	
6	Content Based Filtering in Python	19	
7	Model Based Recommender System	26	

Assignment-1

Objective: User-User Collaborative Filtering using Pearson Similarity Index in Excel.

	A	B	C	D	E	F	G
1	**Name**	**Inshorts(I1)**	**HT(I2)**	**NYT(I3)**	**TOI(I4)**	**BBC(I5)**	**Mean**
2	Aman	5	4	1	4	?	3.5
3	U1	3	1	2	3	3	2.4
4	U2	4	3	4	3	5	3.8
5	U3	3	3	1	5	4	3.2
6							
7			**Mean-Centered Matrix**				
8							
9							
10	**Name**	**Inshorts(I1)**	**HT(I2)**	**NYT(I3)**	**TOI(I4)**	**BBC(I5)**	**Pearson**
11	Aman	1.5	0.5	-2.5	0.5	?	1
12	U1	0.6	-1.4	-0.4	0.6	0.6	0.301511344577764
13	U2	0.2	-0.8	0.2	-0.8	1.2	-0.333333333333333
14	U3	-0.2	-0.2	-2.2	1.8	0.8	0.707106781186548
15							
16	**Prediction for Aman's 5 rating with k = 2:**						
17							
18	4.240213						

Assignment-2

Objective: Item-Item Collaborative Filtering using Cosine Similarity Index in Excel.

	A	B	C	D	E	F	G
1	**Name**	**Inshorts(I1)**	**HT(I2)**	**NYT(I3)**	**TOI(I4)**	**BBC(I5)**	**Mean**
2	Aman	5	4	1	4	?	3.5
3	U1	3	1	2	3	3	2.4
4	U2	4	3	4	3	5	3.8
5	U3	3	3	1	5	4	3.2
6							
7			**Mean-Centered Matrix**				
8							
9							
10	**Name**	**Inshorts(I1)**	**HT(I2)**	**NYT(I3)**	**TOI(I4)**	**BBC(I5)**	
11	Aman	1.5	0.5	-2.5	0.5	?	
12	U1	0.6	-1.4	-0.4	0.6	0.6	
13	U2	0.2	-0.8	0.2	-0.8	1.2	
14	U3	-0.2	-0.2	-2.2	1.8	0.8	
15							
16	**Cosine**	0.42465029	-0.772252272	-0.501883001	0.261156864	1	
17							
18	**Prediction for Aman's item 5 rating with k = 2:**						
19							
20	4.327376528						

Assignment-3

Objective: User-User Collaborative Filtering in Python.

User-User Collaborative Filtering on MovieLens 100K

This notebook implements a fast user-user CF using sparse matrices and sklearn NearestNeighbors, evaluates RMSE/MAE and Precision/Recall@K, and plots results. Run cells top-to-bottom.

```
# 1 — Setup & imports
import os
import zipfile
import urllib.request
from pathlib import Path
import numpy as np
import pandas as pd
from math import sqrt
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.neighbors import NearestNeighbors
from scipy import sparse
import matplotlib.pyplot as plt
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')

# Notebook plotting magic
%matplotlib inline

# Config
DATA_URL = 'https://files.grouplens.org/datasets/movielens/ml-100k.zip'
DATA_DIR = Path('./ml-100k')
RATING_FILE = DATA_DIR / 'u.data'
TEST_FRAC = 0.2
K_NEIGHBORS_FAST = 20
NN_PRECOMPUTE = K_NEIGHBORS_FAST + 1
TOPN = 10
SEED = 42
np.random.seed(SEED)

# 2 — Download MovieLens (if missing) and load ratings
if not DATA_DIR.exists() or not RATING_FILE.exists():
    print('Downloading MovieLens 100K...')
    zip_path = Path('ml-100k.zip')
    urllib.request.urlretrieve(DATA_URL, zip_path)
    with zipfile.ZipFile(zip_path, 'r') as z:
        z.extractall('.')
    zip_path.unlink()
    print('Downloaded.')
else:
    print('MovieLens found at', DATA_DIR)

cols = ['user_id', 'item_id', 'rating', 'timestamp']
ratings = pd.read_csv(RATING_FILE, sep='\t', names=cols, engine='python')
ratings = ratings.drop(columns=['timestamp'])
ratings['user'] = 'u' + ratings['user_id'].astype(str)
```

```
ratings['item'] = 'i' + ratings['item_id'].astype(str)
ratings = ratings[['user', 'item', 'rating']]
print(f"Loaded {len(ratings)} ratings: {ratings['user'].nunique()} users,
{ratings['item'].nunique()} items")
```

```
Downloading MovieLens 100K...
Downloaded.
Loaded 100000 ratings: 943 users, 1682 items
```

```
# 3 — Train/test split per user
def train_test_split_per_user(df, test_frac=0.2, seed=42):
    np.random.seed(seed)
    train_list = []
    test_list = []
    for u, g in df.groupby('user'):
        g_shuf = g.sample(frac=1.0, random_state=seed)
        n = len(g_shuf)
        if n == 0:
            continue
        test_n = max(1, int(np.floor(n * test_frac)))
        test_list.append(g_shuf.iloc[:test_n])
        if n - test_n > 0:
            train_list.append(g_shuf.iloc[test_n:])
    train_df = pd.concat(train_list).reset_index(drop=True) if train_list else
pd.DataFrame(columns=df.columns)
    test_df = pd.concat(test_list).reset_index(drop=True) if test_list else
pd.DataFrame(columns=df.columns)
    return train_df, test_df

train_df, test_df = train_test_split_per_user(ratings, test_frac=TEST_FRAC,
seed=SEED)
print(f"Train ratings: {len(train_df)}, Test ratings: {len(test_df)}")
```

```
Train ratings: 80367, Test ratings: 19633
```

```
# 4 — Build index maps and sparse user-item matrix
all_users = sorted(ratings['user'].unique())
all_items = sorted(ratings['item'].unique())
user_to_idx = {u: i for i, u in enumerate(all_users)}
item_to_idx = {it: i for i, it in enumerate(all_items)}
rows = train_df['user'].map(user_to_idx).astype(int).values
cols = train_df['item'].map(item_to_idx).astype(int).values
data = train_df['rating'].astype(float).values
n_users = len(all_users)
n_items = len(all_items)
R_csr = sparse.csr_matrix((data, (rows, cols)), shape=(n_users, n_items))
print('CSR matrix:', R_csr.shape, ' nnz=', R_csr.nnz)

# Precompute user means and global mean for fallbacks
user_means = train_df.groupby('user')['rating'].mean().to_dict()
global_mean = train_df['rating'].mean()
```

```
CSR matrix: (943, 1682)  nnz= 80367
```

```
# 5 — Fit NearestNeighbors and precompute neighbors
print('Fitting NearestNeighbors...')
nn = NearestNeighbors(n_neighbors=NN_PRECOMPUTE, metric='cosine', algorithm='auto',
n_jobs=-1)
nn.fit(R_csr)
print('Computing kneighbors...')
distances, indices = nn.kneighbors(R_csr, return_distance=True)
similarities = 1.0 - distances

clean_neighbors = [None] * n_users
clean_sims = [None] * n_users
for u_idx in range(n_users):
    neigh = indices[u_idx]
    sims = similarities[u_idx]
    mask = neigh != u_idx
    neigh2 = neigh[mask]
    sims2 = sims[mask]
    neigh2 = neigh2[:K_NEIGHBORS_FAST]
    sims2 = sims2[:K_NEIGHBORS_FAST]
    clean_neighbors[u_idx] = neigh2
    clean_sims[u_idx] = sims2
print('Precomputed neighbors for all users.')

Fitting NearestNeighbors...
Computing kneighbors...
Precomputed neighbors for all users.

# 6 — Precompute item -> raters mapping
item_to_raters = {}
for item, g in train_df.groupby('item'):
    u_idx_arr = g['user'].map(user_to_idx).astype(int).values
    r_arr = g['rating'].astype(float).values
    item_to_raters[item] = (u_idx_arr, r_arr)
print('Built item->raters mapping for', len(item_to_raters), 'items.')

Built item->raters mapping for 1651 items.

# 7 — Fast prediction function
def predict_rating_fast(user, item, shrinkage=0.0):
    if user not in user_to_idx or item not in item_to_idx:
        return np.nan
    u_idx = user_to_idx[user]
    neigh_idxs = clean_neighbors[u_idx]
    neigh_sims = clean_sims[u_idx]
    if item not in item_to_raters:
        return np.nan
    raters_idx, raters_ratings = item_to_raters[item]
    if raters_idx.size == 0:
        return np.nan
    raters_set = set(raters_idx.tolist())
    mask = np.array([int(n) in raters_set for n in neigh_idxs])
    if not mask.any():
        return np.nan
    chosen_neighs = neigh_idxs[mask]
    chosen_sims = neigh_sims[mask]
    rmap = {int(u): float(r) for u, r in zip(raters_idx, raters_ratings)}
    neigh_ratings = np.array([rmap[int(n)] for n in chosen_neighs])
```

```
    denom = np.abs(chosen_sims).sum() + shrinkage
    if denom == 0:
        return np.nan
    pred = (chosen_sims * neigh_ratings).sum() / denom
    return pred

# Quick sanity check on a few known test rows
for _, row in test_df.head(5).iterrows():
    u, it, r = row['user'], row['item'], row['rating']
    print(f"user={u}, item={it}, true={r}, pred={predict_rating_fast(u, it)}")
```

```
user=u1, item=i31, true=3, pred=3.1806436550353943
user=u1, item=i39, true=4, pred=3.005875691025767
user=u1, item=i163, true=4, pred=3.8356973366423697
user=u1, item=i226, true=3, pred=3.2256012679840023
user=u1, item=i169, true=5, pred=4.833496766280945
```

```
# 8 — Evaluate RMSE / MAE
def evaluate_predictions_fast(test_df):
    y_true = []
    y_pred = []
    for _, row in test_df.iterrows():
        u, it, r = row['user'], row['item'], row['rating']
        p = predict_rating_fast(u, it)
        if np.isnan(p):
            p = user_means.get(u, global_mean)
        y_true.append(r)
        y_pred.append(p)
    rmse = sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    return {'rmse': rmse, 'mae': mae, 'n': len(y_true)}

res_fast = evaluate_predictions_fast(test_df)
print('Rating prediction (fast):', res_fast)
```

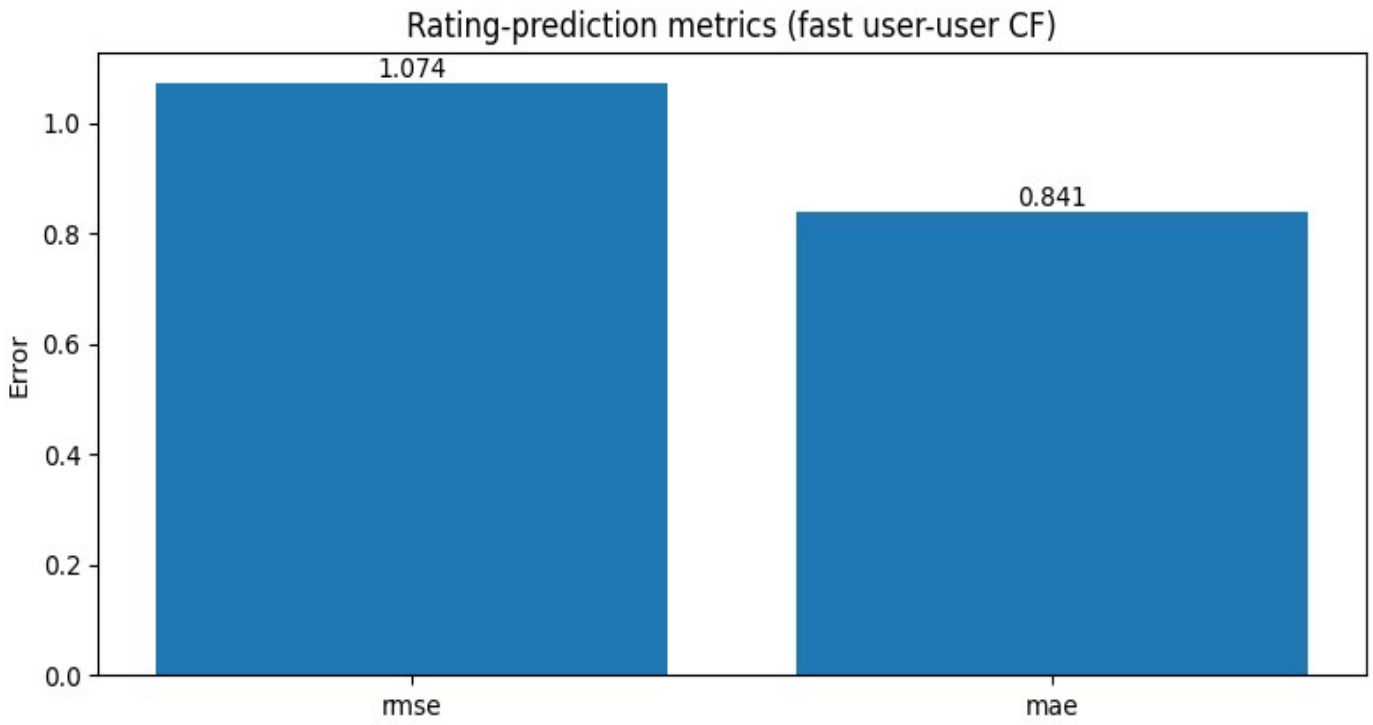
```
Rating prediction (fast): {'rmse': 1.0736103528433045, 'mae': 0.8407547704781173,
'n': 19633}
```

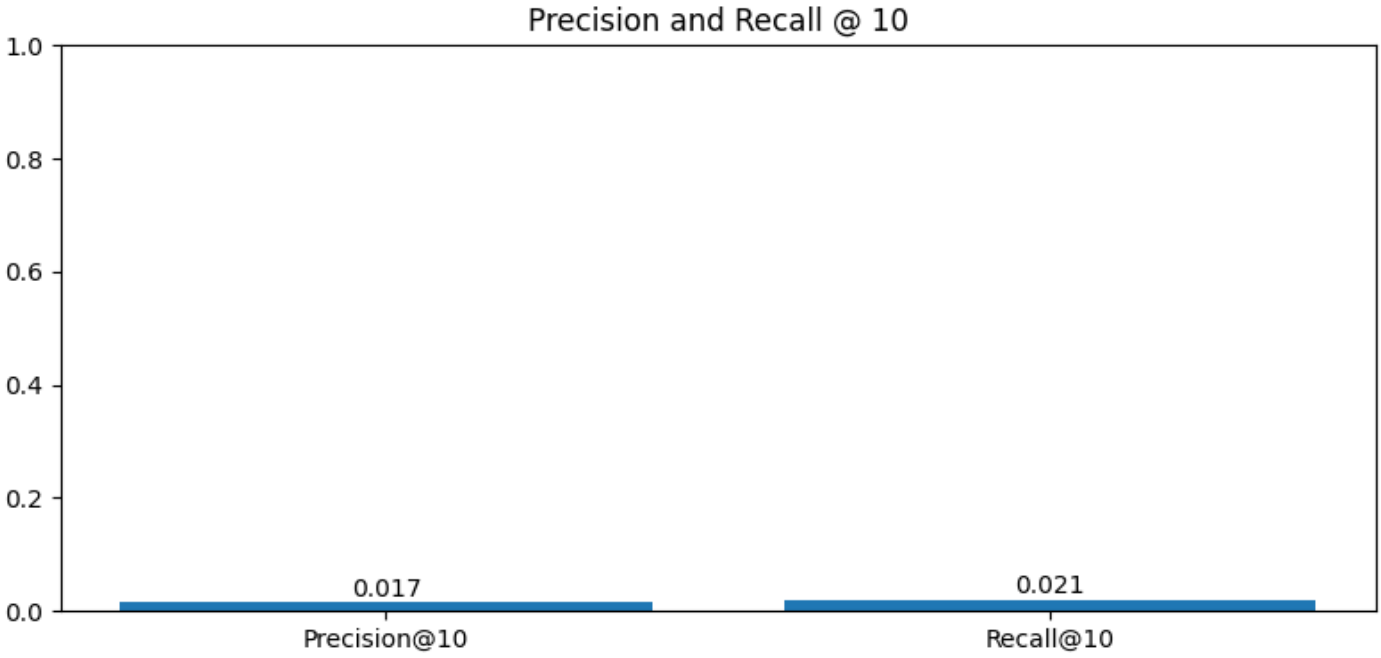
```
# 9 — Top-N: Precision@K, Recall@K
def recommend_top_k_fast(user, top_n=TOPN):
    if user not in user_to_idx:
        return []
    seen_items = set(train_df[train_df['user'] == user]['item'])
    candidate_items = [it for it in all_items if it not in seen_items]
    scores = []
    for it in candidate_items:
        s = predict_rating_fast(user, it)
        scores.append((it, s if not np.isnan(s) else -999))
    scores.sort(key=lambda x: x[1], reverse=True)
    return [it for it, _ in scores[:top_n]]

def precision_recall_at_k_fast(test_df, top_n=TOPN):
    users_common = sorted(set(train_df['user']).intersection(set(test_df['user'])))
    precisions = []
    recalls = []
    for u in users_common:
        true_items = set(test_df[test_df['user'] == u]['item'])
```

```
        if len(true_items) == 0:
            continue
        recs = recommend_top_k_fast(u, top_n=top_n)
        hit = len(set(recs) & true_items)
        precisions.append(hit / top_n)
        recalls.append(hit / len(true_items))
    return {'precision_at_{}'.format(top_n): np.mean(precisions) if precisions else
0.0,
            'recall_at_{}'.format(top_n): np.mean(recalls) if recalls else 0.0,
            'users_eval': len(precisions)}

prec_fast = precision_recall_at_k_fast(test_df, top_n=TOPN)
print('Top-N (fast):', prec_fast)
```

```
Top-N (fast): {'precision_at_10': np.float64(0.01707317073170732), 'recall_at_10':
np.float64(0.020679585449809523), 'users_eval': 943}
```

```
# 10 — Plots: RMSE/MAE and Precision/Recall
metrics_names = ['rmse', 'mae']
metrics_values = [res_fast['rmse'], res_fast['mae']]

plt.figure(figsize=(8,4))
plt.bar(metrics_names, metrics_values)
plt.title('Rating-prediction metrics (fast user-user CF)')
plt.ylabel('Error')
for i, v in enumerate(metrics_values):
    plt.text(i, v + 0.01, f"{v:.3f}", ha='center')
plt.tight_layout()
plt.show()

pr_names = [f'Precision@{TOPN}', f'Recall@{TOPN}']
pr_values = [prec_fast[f'precision_at_{TOPN}'], prec_fast[f'recall_at_{TOPN}']]

plt.figure(figsize=(8,4))
plt.bar(pr_names, pr_values)
plt.title(f'Precision and Recall @ {TOPN}')
plt.ylim(0, 1)
for i, v in enumerate(pr_values):
    plt.text(i, v + 0.01, f"{v:.3f}", ha='center')
plt.tight_layout()
plt.show()
```






```
# 11 — Save evaluation summary and example recommendations
summary = pd.DataFrame([{'method': 'user-user (fast, cosine NN)', **res_fast,
**prec_fast}])
summary.to_csv('uu_cf_movielens_fast_eval_summary.csv', index=False)
print('Saved summary to uu_cf_movielens_fast_eval_summary.csv')

# Show example recommendations for first 5 users
```

```
for u in all_users[:5]:
    recs = recommend_top_k_fast(u, top_n=5)
    print(f'user={u}, recs={recs}')

Saved summary to uu_cf_movielens_fast_eval_summary.csv
user=u1, recs=['i1093', 'i1111', 'i1114', 'i613', 'i675']
user=u10, recs=['i109', 'i1195', 'i1199', 'i1428', 'i305']
user=u100, recs=['i1025', 'i1175', 'i337', 'i345', 'i361']
user=u101, recs=['i10', 'i1277', 'i173', 'i216', 'i249']
user=u102, recs=['i1019', 'i1119', 'i116', 'i124', 'i137']
```

Assignment-4

Objective: Item-Item Collaborative Filtering in Python.

Item-Item Collaborative Filtering on MovieLens 100K

This notebook implements a fast item-item CF using sparse matrices and sklearn NearestNeighbors, evaluates RMSE/MAE and Precision/Recall@K, and plots results.

```
# 1. Imports & configuration
import os, zipfile, urllib.request
from pathlib import Path
import numpy as np
import pandas as pd
from math import sqrt
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy import sparse
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
# %matplotlib inline

DATA_URL = 'https://files.grouplens.org/datasets/movielens/ml-100k.zip'
DATA_DIR = Path('./ml-100k')
RATING_FILE = DATA_DIR / 'u.data'
TEST_FRAC = 0.2
K_NEIGHBORS = 20
NN_PRECOMPUTE = K_NEIGHBORS + 1
TOPN = 10
SEED = 42
np.random.seed(SEED)

print('Config: K=', K_NEIGHBORS, 'TOPN=', TOPN)

Config: K= 20 TOPN= 10

# 2. Download and load MovieLens 100K
if not DATA_DIR.exists() or not RATING_FILE.exists():
    print('Downloading MovieLens 100K...')
    zip_path = Path('ml-100k.zip')
    urllib.request.urlretrieve(DATA_URL, zip_path)
    with zipfile.ZipFile(zip_path, 'r') as z:
        z.extractall('.')
    zip_path.unlink()
    print('Downloaded and extracted.')
else:
    print('Dataset found at', DATA_DIR)

cols = ['user_id','item_id','rating','timestamp']
ratings = pd.read_csv(RATING_FILE, sep='\t', names=cols, engine='python')
ratings = ratings.drop(columns=['timestamp'])
ratings['user'] = 'u' + ratings['user_id'].astype(str)
ratings['item'] = 'i' + ratings['item_id'].astype(str)
ratings = ratings[['user','item','rating']]
```

```
print(f"Loaded {len(ratings)} ratings: {ratings['user'].nunique()} users,
{ratings['item'].nunique()} items")
```

```
Dataset found at ml-100k
Loaded 100000 ratings: 943 users, 1682 items
```

```
# 3. Train-test split (per-user)
def train_test_split_per_user(df, test_frac=0.2, seed=42):
    np.random.seed(seed)
    train_rows = []
    test_rows = []
    for u, g in df.groupby('user'):
        g_shuf = g.sample(frac=1.0, random_state=seed)
        n = len(g_shuf)
        if n == 0:
            continue
        test_n = max(1, int(np.floor(n * test_frac)))
        test_rows.append(g_shuf.iloc[:test_n])
        if n - test_n > 0:
            train_rows.append(g_shuf.iloc[test_n:])
    train_df = pd.concat(train_rows).reset_index(drop=True) if train_rows else
pd.DataFrame(columns=df.columns)
    test_df = pd.concat(test_rows).reset_index(drop=True) if test_rows else
pd.DataFrame(columns=df.columns)
    return train_df, test_df

train_df, test_df = train_test_split_per_user(ratings, test_frac=TEST_FRAC,
seed=SEED)
print('Train ratings:', len(train_df), 'Test ratings:', len(test_df))
```

```
Train ratings: 80367 Test ratings: 19633
```

```
# 4. Build index maps and sparse CSR user-item matrix
all_users = sorted(ratings['user'].unique())
all_items = sorted(ratings['item'].unique())
user_to_idx = {u:i for i,u in enumerate(all_users)}
item_to_idx = {it:i for i,it in enumerate(all_items)}
idx_to_user = {i:u for u,i in user_to_idx.items()}
idx_to_item = {i:it for it,i in item_to_idx.items()}

rows = train_df['user'].map(user_to_idx).astype(int).values
cols = train_df['item'].map(item_to_idx).astype(int).values
data = train_df['rating'].astype(float).values
n_users = len(all_users)
n_items = len(all_items)
R_csr = sparse.csr_matrix((data, (rows, cols)), shape=(n_users, n_items))
print('CSR matrix built:', R_csr.shape, ' nnz=', R_csr.nnz)

# Precompute user means and global mean for fallbacks
user_means = train_df.groupby('user')['rating'].mean().to_dict()
global_mean = train_df['rating'].mean()
print('Global mean rating (train):', global_mean)
```

```
CSR matrix built: (943, 1682)  nnz= 80367
Global mean rating (train): 3.5313499321860964
```

```
# 5. Compute item-item similarities using NearestNeighbors (cosine)
print('Fitting NearestNeighbors on item vectors...')
R_items = R_csr.T  # items x users
nn_item = NearestNeighbors(n_neighbors=NN_PRECOMPUTE, metric='cosine', n_jobs=-1)
nn_item.fit(R_items)
dist_i, idx_i = nn_item.kneighbors(R_items, return_distance=True)
sim_i = 1.0 - dist_i

# Clean neighbor lists: remove self and keep top-K
item_neighbors = [None] * n_items
item_sims = [None] * n_items
for i_idx in range(n_items):
    neigh = idx_i[i_idx]
    sims = sim_i[i_idx]
    mask = neigh != i_idx
    neigh2 = neigh[mask][:K_NEIGHBORS]
    sims2 = sims[mask][:K_NEIGHBORS]
    item_neighbors[i_idx] = neigh2
    item_sims[i_idx] = sims2
print('Item-item neighbors computed for', n_items, 'items')

Fitting NearestNeighbors on item vectors...
Item-item neighbors computed for 1682 items

# 6. Precompute user -> (item_idx, rating) mapping
user_to_items = {}
for u, g in train_df.groupby('user'):
    item_idx_arr = g['item'].map(item_to_idx).astype(int).values
    ratings_arr = g['rating'].astype(float).values
    user_to_items[u] = (item_idx_arr, ratings_arr)
print('Built user->items for', len(user_to_items), 'users')

Built user->items for 943 users

# 7. Predict function (item-item), analogous to user-user predictor
import numpy as np

def predict_item_item(user, item, k=K_NEIGHBORS, shrinkage=0.0):
    """Predict rating for (user, item) using item-item CF: weighted average of
user's ratings on similar items"""
    if user not in user_to_items or item not in item_to_idx:
        return np.nan
    i_idx = item_to_idx[item]
    neigh_idxs = item_neighbors[i_idx]
    neigh_sims = item_sims[i_idx]
    # user's rated items
    rated_idx, rated_vals = user_to_items[user]
    if rated_idx.size == 0:
        return np.nan
    rated_set = set(rated_idx.tolist())
    mask = np.array([int(n) in rated_set for n in neigh_idxs])
    if not mask.any():
        return np.nan
    chosen_items = neigh_idxs[mask]
    chosen_sims = neigh_sims[mask]
    rmap = {int(it): float(r) for it, r in zip(rated_idx, rated_vals)}
    neigh_ratings = np.array([rmap[int(it)] for it in chosen_items])
```

```
    denom = np.abs(chosen_sims).sum() + shrinkage
    if denom == 0:
        return np.nan
    pred = (chosen_sims * neigh_ratings).sum() / denom
    return pred

# quick sanity checks
for _, row in test_df.head(5).iterrows():
    u, it, r = row['user'], row['item'], row['rating']
    print('user', u, 'item', it, 'true', r, 'pred', predict_item_item(u, it))
```

```
user u1 item i31 true 3 pred 4.200947019986595
user u1 item i39 true 4 pred 4.070163121944019
user u1 item i163 true 4 pred 4.450715667168242
user u1 item i226 true 3 pred 3.6460405739948825
user u1 item i169 true 5 pred 4.400948565805868
```

```
# 8. Evaluate rating prediction (RMSE / MAE)
y_true = []
y_pred = []
for _, row in test_df.iterrows():
    u, it, r = row['user'], row['item'], row['rating']
    p = predict_item_item(u, it)
    if np.isnan(p):
        p = user_means.get(u, global_mean)
    y_true.append(r)
    y_pred.append(p)

rmse = sqrt(mean_squared_error(y_true, y_pred))
mae = mean_absolute_error(y_true, y_pred)
print('Item-Item RMSE:', rmse, 'MAE:', mae)
```

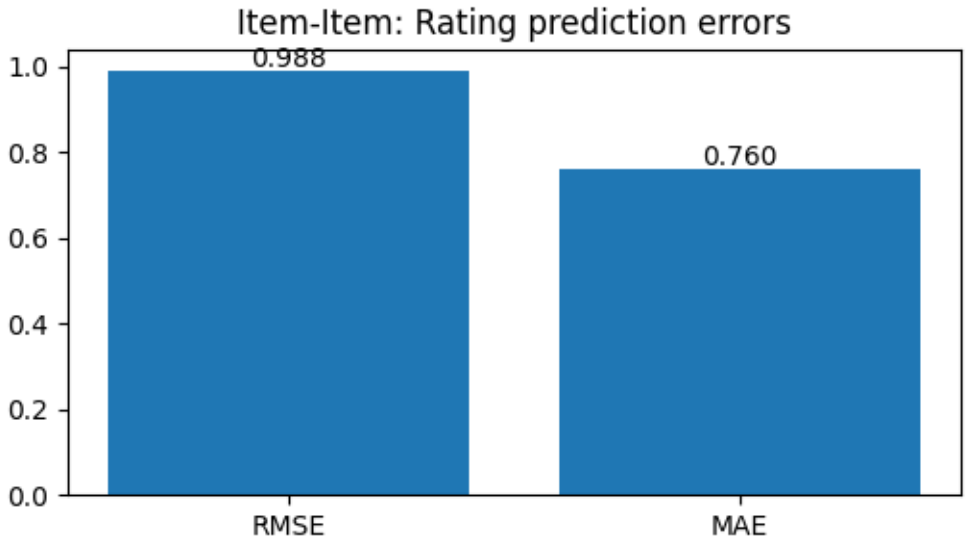
```
Item-Item RMSE: 0.9879396716330665 MAE: 0.7599870584145011
```

```
# 9. Top-N evaluation: Precision@K, Recall@K

def recommend_top_k_item(user, top_n=TOPN):
    if user not in user_to_items:
        return []
    seen_items = set(train_df[train_df['user'] == user]['item'])
    candidates = [it for it in all_items if it not in seen_items]
    scores = []
    for it in candidates:
        s = predict_item_item(user, it)
        scores.append((it, s if not np.isnan(s) else -999))
    scores.sort(key=lambda x: x[1], reverse=True)
    return [it for it, _ in scores[:top_n]]

precisions = []
recalls = []
for u in sorted(set(train_df['user']).intersection(set(test_df['user']))):
    true_items = set(test_df[test_df['user'] == u]['item'])
    if len(true_items) == 0:
        continue
    recs = recommend_top_k_item(u, top_n=TOPN)
    hit = len(set(recs) & true_items)
    precisions.append(hit / TOPN)
```

```
    recalls.append(hit / len(true_items))

print('Precision@{}:'.format(TOPN), np.mean(precisions), 'Recall@{}:'.format(TOPN),
np.mean(recalls))
```

```
Precision@10: 0.011558854718981973 Recall@10: 0.008225605562151737
```

```
# 10. Plots
# Bar plot for RMSE/MAE (single values)
plt.figure(figsize=(6,3))
plt.bar(['RMSE','MAE'], [rmse, mae])
plt.title('Item-Item: Rating prediction errors')
for i,v in enumerate([rmse, mae]): plt.text(i, v+0.01, f"{v:.3f}", ha='center')
plt.show()

# Example: Precision & Recall values printed above
```



Assignment-5

Objective: Content Based Filtering in Excel.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1		baseball	economic	politics	Europe	Asia	soccer	war	security	shopping	family		num-attr		User 1	User 2		Pred1	Pred2
2	doc1	1	0	1	0	1	1	0	0	0	1		5		1	-1		4	-4
3	doc2	0	1	1	1	0	0	0	1	0	0		4		-1	1		-4	10
4	doc3	0	0	0	1	1	1	0	0	0	0		3					2	0
5	doc4	0	0	1	1	0	0	1	1	0	0		4			1		-3	8
6	doc5	0	1	0	0	0	0	0	0	1	1		3					-1	1
7	doc6	1	0	0	1	0	0	0	0	0	0		2		1			3	1
8	doc7	0	0	0	0	0	0	0	1	0	1		2					-1	2
9	doc8	0	0	1	1	0	0	1	0	0	1		4					-2	4
10	doc9	0	0	0	0	0	1	0	0	1	0		2					3	-2
11	doc10	0	1	0	0	1	0	1	0	0	0		3					-3	1
12	doc11	0	0	1	0	1	0	0	0	1	0		3					0	1
13	doc12	1	0	0	0	0	1	1	0	0	0		3			-1		4	-4
14	doc13	0	0	1	1	1	0	0	1	0	0		4					-2	7
15	doc14	0	1	1	1	0	0	0	0	1	0		4					-2	7
16	doc15	0	0	0	1	0	1	1	1	0	0		4					0	4
17	doc16	1	0	0	0	0	1	0	0	1	0		3		1			6	-4
18	doc17	0	1	1	1	0	0	0	1	0	0		4			1		-4	10
19	doc18	0	0	0	1	0	0	0	0	1	0		2					1	3
20	doc19	0	1	1	0	1	0	1	0	0	1		5		-1			-4	2
21	doc20	0	0	1	1	0	0	1	0	1	0		4					-1	5
22																			
23	DF	4	6	10	11	6	6	7	6	7	5								
24																			
25																			
26	User Profiles																		
27	User1	3	-2	-1	0	0	2	-1	-1	1	0								
28	User2	-2	2	2	3	-1	-2	0	3	0	-1								

	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG
1												Pred1	Pred2
2	0.447213595499958	0	0.447213595499958	0	0.447213595499958	0.447213595499958	0	0	0	0.447213595499958		1.009018774776118	0.845577386244385
3	0	0.5	0.5	0.5	0	0	0	0.5	0	0		0.870053407156705	2.526393202250002
4	0	0	0	0.577350269189626	0.577350269189626	0.577350269189626	0	0	0	0		0.711105378949545	0.016294209956783
5	0	0	0.5	0.5	0	0	0.5	0.5	0	0		0.620053407156705	1.987718067655211
6	0	0.577350269189626	0	0	0	0	0	0	0.577350269189626	0.577350269189626		0.213540691008641	0.319151379442465
7	0.707106781186548	0	0	0.707106781186548	0	0	0	0	0	0		1.37092266588743	0.33618411529912
8	0	0	0	0	0	0	0	0.707106781186548	0	0.707106781186548		0.353553390593274	0.744432405762983
9	0	0	0.5	0.5	0	0	0.5	0	0	0.5		0.370053407156705	1.014111269905231
10	0	0	0	0	0	0.707106781186548	0	0	0.707106781186548	0		1.132724346944560	0.724476056480701
11	0	0.577350269189626	0	0	0.577350269189626	0	0.577350269189626	0	0	0		0.805072914089135	0.274493180703944
12	0	0	0.577350269189626	0	0.577350269189626	0	0	0	0.577350269189626	0		0.044658198738521	0.349627624290116
13	0.577350269189626	0	0	0	0	0.577350269189626	0.577350269189626	0	0	0		1.333113846877690	-1.227722644899510
14	0	0	0.5	0.5	0.5	0	0	0	0.5	0		0.396446609406726	1.802786404500040
15	0	0.5	0.5	0.5	0	0	0	0	0	0.5		0.331378272561892	1.776393202250020
16	0	0	0	0.5	0	0.5	0.5	0.5	0	0		0.142228525188087	0.949042933060395
17	0.577350269189626	0	0	0	0	0.577350269189626	0	0	0.577350269189626	0		1.924646069958190	-1.183064446160990
18	0	0.5	0.5	0.5	0	0	0	0.5	0	0		0.870053407156705	2.526393202250020
19	0	0	0	0.707106781186548	0	0	0	0	0.707106781186548	0		0.554694899870589	1.060660171779820
20	0	0.447213595499958	0.447213595499958	0	0.447213595499958	0	0.447213595499958	0	0	0.447213595499958		0.847213595499958	0.483441896752713
21	0	0	0.5	0.5	0	0	0.5	0	0.5	0		0.081378272561892	1.237718067655210
22													
23													
24													
25													
26													
27	1.731670645876130	0.947213595499958	-0.5	0.207106781186547	0	1.024563864689580	0.447213595499958	-0.5	0.577350269189626	0			
28	-1.024563864689580	1	1.052786404500040	1.5	0.447213595499958	-1.024563864689580	0.077350269189626	1.5	0	0.447213595499958			

	A	B	C	D	E	F	G	H	I	J	K
25											
26	User Profiles										
27	User1	3	-2	-1	0	0	2	-1	-1	1	0
28	User2	-2	2	2	3	-1	-2	0	3	0	-1
29											
30	IDF	0.25	0.16667	0.1	0.09091	0.16667	0.16667	0.14286	0.16667	0.14286	0.2
31											
32	Pred1	Pred1									
33	0.24761	-0.21717									
34	-0.13619	0.32915									
35	0.10946	-0.06289									
36	-0.0892	0.2403									
37	-0.04353	0.04459									
38	0.31943	-0.0847									
39	-0.05893	0.11353									
40	-0.04753	0.07057									
41	0.17907	-0.12075									
42	-0.12803	0.04681									
43	0.01875	0.01775									
44	0.31165	-0.25285									
45	-0.05725	0.20855									
46	-0.05328	0.20415									
47	0.02118	0.10228									
48	0.39615	-0.24647									
49	-0.13619	0.32915									
50	0.07163	0.09642									
51	-0.12153	0.04334									
52	-0.00629	0.1153									

Assignment-6

Objective: Content Based Filtering in Python.

Content-Based Filtering on MovieLens 100K

This notebook implements a content-based recommender using MovieLens 100K features (titles + genres). It trains on a per-user train/test split, builds item feature vectors (TF-IDF on titles + genre one-hot), constructs user profiles from rated items, predicts ratings using similarity-weighted ratings from user's rated items, and evaluates using RMSE/MAE and Precision@K/Recall@K. Plots are included.

```
# 1 — Imports & configuration
import os, zipfile, urllib.request
from pathlib import Path
import numpy as np, pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import mean_squared_error, mean_absolute_error
from math import sqrt
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Notebook magic (uncomment when running in a notebook)
# %matplotlib inline

DATA_URL = "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
DATA_DIR = Path("./ml-100k")
MOVIE_FILE = DATA_DIR / "u.item"   # movie metadata
RATING_FILE = DATA_DIR / "u.data"
TEST_FRAC = 0.2
EVAL_TOPN = 10
EVAL_KS = [5, 10, 20]  # number of similar rated items to use in
prediction/recommendation
SEED = 42
np.random.seed(SEED)

print('Config: TEST_FRAC=', TEST_FRAC, 'TOPN=', EVAL_TOPN, 'EVAL_KS=', EVAL_KS)

Config: TEST_FRAC= 0.2 TOPN= 10 EVAL_KS= [5, 10, 20]

# 2 — Download MovieLens 100K (if needed) and load ratings & movie metadata
if not DATA_DIR.exists() or not RATING_FILE.exists() or not MOVIE_FILE.exists():
    print("Downloading MovieLens 100K...")
    zip_path = Path("ml-100k.zip")
    urllib.request.urlretrieve(DATA_URL, zip_path)
    import zipfile
    with zipfile.ZipFile(zip_path, "r") as z:
        z.extractall(".")
    zip_path.unlink()
    print("Downloaded and extracted.")
else:
    print("Dataset found at", DATA_DIR)
```

```
# Load ratings (u.data): user_id \t item_id \t rating \t timestamp
cols = ["user_id", "item_id", "rating", "timestamp"]
ratings = pd.read_csv(RATING_FILE, sep="\t", names=cols, engine="python")
ratings = ratings.drop(columns=["timestamp"])
ratings["user"] = "u" + ratings["user_id"].astype(str)
ratings["item"] = "i" + ratings["item_id"].astype(str)
ratings = ratings[["user", "item", "rating"]]
print(f"Loaded {len(ratings)} ratings: {ratings['user'].nunique()} users,
{ratings['item'].nunique()} items")

# Load movie metadata (u.item): movie id | title | release date | video release
date | URL | genres (19 flags)
# u.item is pipe-separated; some titles contain special characters
movie_cols = ['movie_id','title','release_date','video_release_date','url'] +
[f'genre_{i}' for i in range(19)]
movies = pd.read_csv(MOVIE_FILE, sep='|', names=movie_cols, encoding='latin-1',
engine='python')
movies['item'] = 'i' + movies['movie_id'].astype(str)
movies = movies[['item','title'] + [c for c in movie_cols if
c.startswith('genre_')]]
# Build genres string from genre flags
genre_cols = [c for c in movies.columns if c.startswith('genre_')]
def genres_to_text(row):
    genres = [str(i) for i, c in enumerate(genre_cols) if row[c] == 1]
    return ' '.join(genres)
movies['genres_text'] = movies.apply(genres_to_text, axis=1)

print('Loaded', len(movies), 'movies')

Downloading MovieLens 100K...
Downloaded and extracted.
Loaded 100000 ratings: 943 users, 1682 items
Loaded 1682 movies

# 3 — Train/test split per user (leave at least 1 per user for test)
def train_test_split_per_user(df, test_frac=TEST_FRAC, seed=SEED):
    np.random.seed(seed)
    train_rows = []
    test_rows = []
    for u, g in df.groupby('user'):
        g_shuf = g.sample(frac=1.0, random_state=seed)
        n = len(g_shuf)
        if n == 0: continue
        test_n = max(1, int(np.floor(n * test_frac)))
        test_rows.append(g_shuf.iloc[:test_n])
        if n - test_n > 0:
            train_rows.append(g_shuf.iloc[test_n:])
    train_df = pd.concat(train_rows).reset_index(drop=True)
    test_df = pd.concat(test_rows).reset_index(drop=True)
    return train_df, test_df

train_df, test_df = train_test_split_per_user(ratings)
print('Train ratings:', len(train_df), 'Test ratings:', len(test_df))

Train ratings: 80367 Test ratings: 19633
```

```
# 4 — Build item feature vectors: TF-IDF on titles + genre one-hot concatenation
from sklearn.preprocessing import normalize

# Merge train/test item lists with movies to ensure coverage
all_items = sorted(ratings['item'].unique())
movies_sub = movies[movies['item'].isin(all_items)].set_index('item')

# Prepare textual feature: title (TF-IDF)
titles = movies_sub['title'].fillna('').astype(str).values
tfidf = TfidfVectorizer(stop_words='english', max_features=2000)
title_feats = tfidf.fit_transform(titles)  # shape: (n_items, n_title_feats)

# Prepare genre features (19 flags -> as numeric columns)
genre_flags = movies_sub[genre_cols].fillna(0).astype(int).values  # shape:
(n_items, 19)

# Combine features: horizontal stack (sparse + dense -> convert dense to sparse)
from scipy.sparse import hstack, csr_matrix
genre_sparse = csr_matrix(genre_flags)
item_features = hstack([title_feats, genre_sparse]).tocsr()  # final item feature
matrix

print('Item features shape:', item_features.shape)

Item features shape: (1682, 2019)

# 5 — Precompute item-item similarity matrix (cosine on feature vectors)
# For ~1682 items this is fine (dense similarity matrix ~ (1682^2) floats)
print('Computing item-item cosine similarity (this may take a few seconds)...')
t0 = __import__('time').time()
item_sim_matrix = cosine_similarity(item_features, dense_output=True)
t1 = __import__('time').time()
print(f'Computed item-item similarity in {t1-t0:.2f}s')

Computing item-item cosine similarity (this may take a few seconds)...
Computed item-item similarity in 0.06s

# 6 — Precompute mappings for fast prediction
# Map items to index in our item_features ordering
item_list = movies_sub.index.tolist()  # same order as title_feats input
item_to_idx = {it: idx for idx, it in enumerate(item_list)}
idx_to_item = {idx: it for it, idx in item_to_idx.items()}

# user -> (rated_item_indices, ratings)
user_profiles = {}
for u, g in train_df.groupby('user'):
    # only keep rated items that exist in item_to_idx
    idxs = [item_to_idx[it] for it in g['item'] if it in item_to_idx]
    ratings_arr = [float(r) for it, r in zip(g['item'], g['rating']) if it in
item_to_idx]
    if len(idxs) == 0:
        continue
    user_profiles[u] = (np.array(idxs, dtype=int), np.array(ratings_arr,
dtype=float))
```

```
print('Built user_profiles for', len(user_profiles), 'users')
```

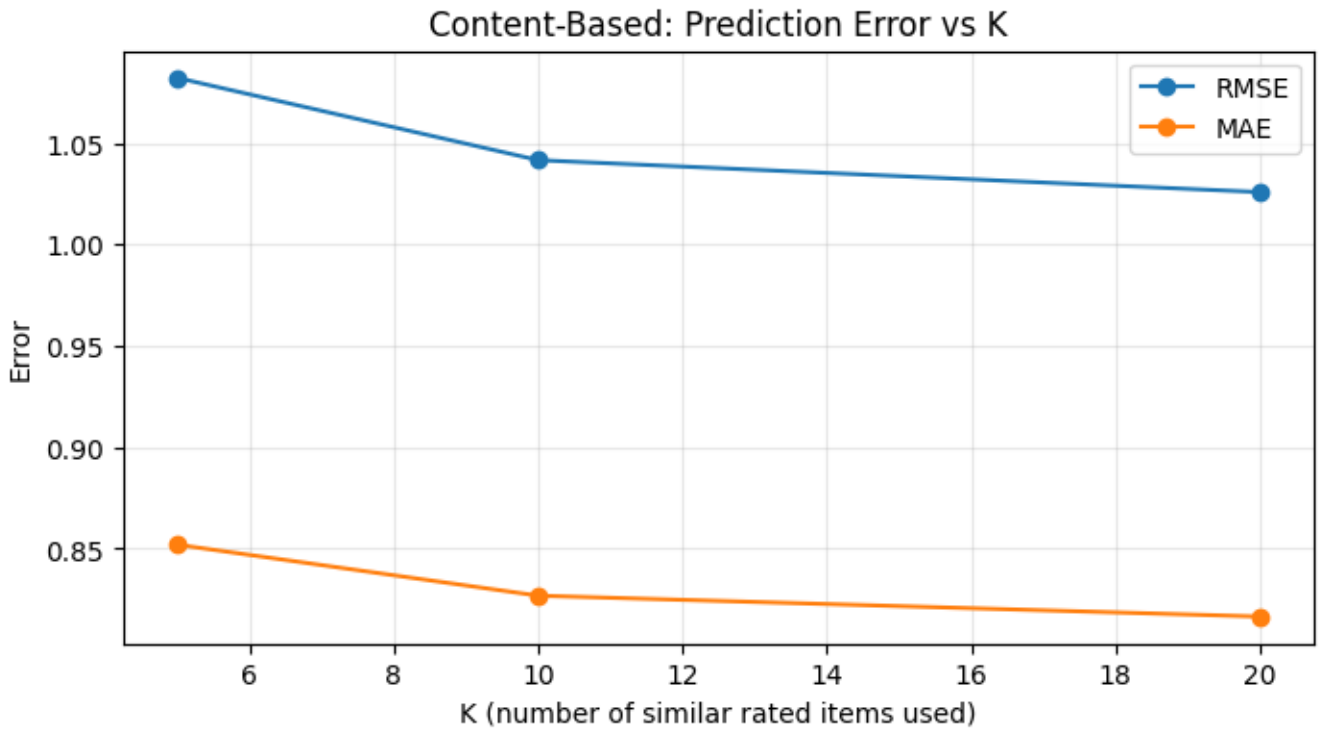
```
Built user_profiles for 943 users
```

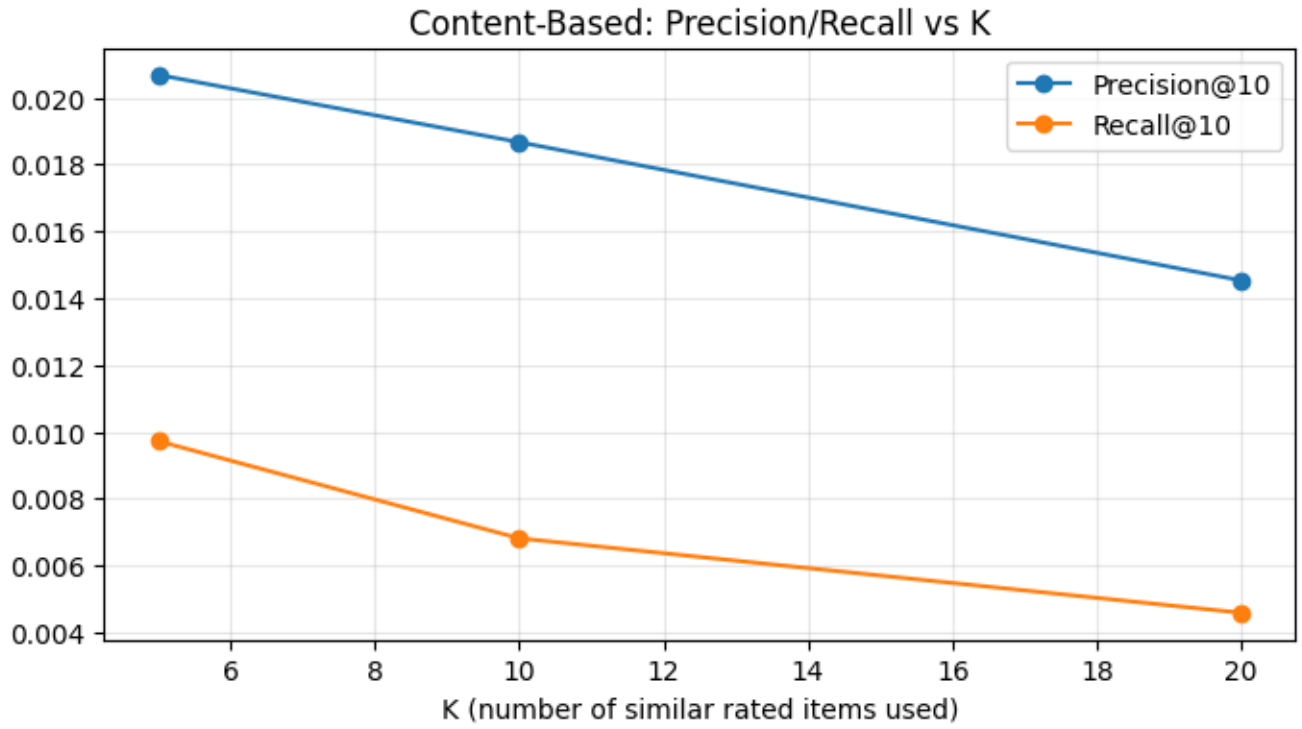
```
# 7 — Prediction: similarity-weighted rating from user's rated items (content-
based)
# For a target (user, item): compute similarities between target item and items the
user rated,
# pick top-K most similar rated items, and do weighted average of the user's
ratings (weights=similarities).
def predict_cbf(user, item, K=10, shrinkage=0.0):
    if user not in user_profiles or item not in item_to_idx:
        return np.nan
    rated_idxs, rated_ratings = user_profiles[user]
    target_idx = item_to_idx[item]
    sims = item_sim_matrix[target_idx, rated_idxs]  # similarities to items user
rated
    # if all zeros or no overlap, return NaN
    if sims.size == 0:
        return np.nan
    # choose top-K similar rated items
    top_k_idx = np.argsort(-sims)[:K]
    chosen_sims = sims[top_k_idx]
    chosen_ratings = rated_ratings[top_k_idx]
    denom = np.abs(chosen_sims).sum() + shrinkage
    if denom == 0:
        return np.nan
    pred = (chosen_sims * chosen_ratings).sum() / denom
    return float(pred)

# 8 — Evaluate rating prediction (RMSE, MAE) for different K values
from sklearn.metrics import mean_squared_error, mean_absolute_error

def evaluate_rating_pred(test_df, K=10):
    y_true = []
    y_pred = []
    for _, row in test_df.iterrows():
        u, it, r = row['user'], row['item'], row['rating']
        p = predict_cbf(u, it, K=K)
        # fallback to user's mean in train or global mean
        if np.isnan(p):
            p = train_df[train_df['user'] == u]['rating'].mean() if
len(train_df[train_df['user'] == u]) > 0 else train_df['rating'].mean()
        y_true.append(r); y_pred.append(p)
    rmse = sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    return {'rmse': rmse, 'mae': mae, 'n': len(y_true)}

results = []
for K in EVAL_KS:
    res = evaluate_rating_pred(test_df, K=K)
    print(f'K={K}: RMSE={res["rmse"]:.4f}, MAE={res["mae"]:.4f}')
    results.append((K, res['rmse'], res['mae']))

res_df = pd.DataFrame(results, columns=['K','RMSE','MAE'])
```

```
K=5: RMSE=1.0820, MAE=0.8520
```

```
K=10: RMSE=1.0415, MAE=0.8269
K=20: RMSE=1.0258, MAE=0.8166

# 9 — Top-N evaluation: Precision@K and Recall@K for content-based recommendations
def recommend_top_n_cbf(user, K=10, top_n=EVAL_TOPN):
    if user not in user_profiles:
        return []
    rated_idxs, rated_ratings = user_profiles[user]
    seen = set([idx_to_item[i] for i in rated_idxs])
    # score all items user hasn't rated by similarity-weighted rating
    scores = []
    for it in item_list:
        if it in seen: continue
        p = predict_cbf(user, it, K=K)
        scores.append((it, p if not np.isnan(p) else -999))
    scores.sort(key=lambda x: x[1], reverse=True)
    return [it for it, _ in scores[:top_n]]

def precision_recall_cbf(test_df, K=10, top_n=EVAL_TOPN):
    precisions = []
    recalls = []
    users_eval = 0
    for u in sorted(set(train_df['user']).intersection(set(test_df['user']))):
        true_items = set(test_df[test_df['user'] == u]['item'])
        if len(true_items) == 0: continue
        recs = recommend_top_n_cbf(u, K=K, top_n=top_n)
        hit = len(set(recs) & true_items)
        precisions.append(hit / top_n)
        recalls.append(hit / len(true_items))
        users_eval += 1
    return {'precision': float(np.mean(precisions) if precisions else 0.0),
            'recall': float(np.mean(recalls) if recalls else 0.0),
            'users_eval': users_eval}

pr_results = []
for K in EVAL_KS:
    pr = precision_recall_cbf(test_df, K=K, top_n=EVAL_TOPN)
    print(f'K={K}: Precision@{EVAL_TOPN}={pr["precision"]:.4f}, Recall@{EVAL_TOPN}
={pr["recall"]:.4f}')
    pr_results.append((K, pr['precision'], pr['recall']))

pr_df = pd.DataFrame(pr_results, columns=['K','Precision','Recall'])

K=5: Precision@10=0.0207, Recall@10=0.0097
K=10: Precision@10=0.0187, Recall@10=0.0068
K=20: Precision@10=0.0145, Recall@10=0.0046

# 10 — Plots: RMSE/MAE vs K and Precision/Recall vs K
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))
plt.plot(res_df['K'], res_df['RMSE'], marker='o', label='RMSE')
plt.plot(res_df['K'], res_df['MAE'], marker='o', label='MAE')
plt.xlabel('K (number of similar rated items used)')
plt.ylabel('Error')
plt.title('Content-Based: Prediction Error vs K')
plt.legend()
```

```
plt.grid(alpha=0.3)
plt.show()

plt.figure(figsize=(8,4))
plt.plot(pr_df['K'], pr_df['Precision'], marker='o',
label=f'Precision@{EVAL_TOPN}')
plt.plot(pr_df['K'], pr_df['Recall'], marker='o', label=f'Recall@{EVAL_TOPN}')
plt.xlabel('K (number of similar rated items used)')
plt.title('Content-Based: Precision/Recall vs K')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```






```
# 11 — Save evaluation summary and show examples
res_df.to_csv('cbf_rating_eval_summary.csv', index=False)
pr_df.to_csv('cbf_topn_eval_summary.csv', index=False)
print('Saved cbf_rating_eval_summary.csv and cbf_topn_eval_summary.csv')

# Show sample recommendations for a few users
for u in list(user_profiles.keys())[:5]:
    print('\\nUser', u)
    print('  Top-5 CBF recs:', recommend_top_n_cbf(u, K=EVAL_KS[1], top_n=5))
```

```
Saved cbf_rating_eval_summary.csv and cbf_topn_eval_summary.csv
\nUser u1
  Top-5 CBF recs: ['i488', 'i1476', 'i720', 'i857', 'i1594']
\nUser u10
  Top-5 CBF recs: ['i914', 'i646', 'i1669', 'i327', 'i1064']
\nUser u100
  Top-5 CBF recs: ['i1373', 'i32', 'i48', 'i75', 'i119']
\nUser u101
  Top-5 CBF recs: ['i593', 'i611', 'i1064', 'i1065', 'i1187']
\nUser u102
  Top-5 CBF recs: ['i1476', 'i1582', 'i488', 'i1232', 'i1123']
```

Assignment-7

Objective: Model Based Recommender System.

Neural Network Model-Based Recommender (Neural Collaborative Filtering)

This notebook trains a neural-network-based recommender (Neural Collaborative Filtering) on MovieLens 100K using PyTorch. It includes:

- data loading and per-user train/test split
- PyTorch Dataset/DataLoader
- NCF model (user & item embeddings + MLP)
- training loop with MSE loss
- evaluation: RMSE/MAE and Top-N Precision@K / Recall@K
- plots for training loss and metrics

If PyTorch is not installed, the notebook will attempt to install it (CPU-only). Run cells top-to-bottom in Jupyter.

```
# 1. Setup: imports and configuration
import sys, os, math, time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from math import sqrt
import warnings
warnings.filterwarnings('ignore')

# Try importing torch; if not installed, install CPU-only torch
try:
    import torch
    import torch.nn as nn
    import torch.optim as optim
    from torch.utils.data import Dataset, DataLoader
    TORCH_AVAILABLE = True
except Exception as e:
    TORCH_AVAILABLE = False
    print('PyTorch not available. The next cell will install a CPU-only build if
running in an environment with pip.')

# Config
DATA_URL = 'https://files.grouplens.org/datasets/movielens/ml-100k.zip'
DATA_DIR = Path('./ml-100k')
RATING_FILE = DATA_DIR / 'u.data'
TEST_FRAC = 0.2
EMBED_DIM = 32
MLP_LAYERS = [64,32,16]
LR = 1e-3
BATCH_SIZE = 1024
EPOCHS = 8
```

```
TOPN = 10
SEED = 42
DEVICE = 'cuda' if torch.cuda.is_available() and TORCH_AVAILABLE else 'cpu' if
TORCH_AVAILABLE else 'cpu'
np.random.seed(SEED)

print('DEVICE =', DEVICE)
```

```
DEVICE = cpu
```

```
# Install PyTorch (CPU-only) if missing. This may take a minute.
import sys
if not 'torch' in sys.modules:
    try:
        print('Installing torch...')
        # CPU-only install via pip. In some environments an appropriate wheel is
needed.
        !pip install --quiet torch torchvision --index-url
https://download.pytorch.org/whl/cpu
        import torch
        import torch.nn as nn
        import torch.optim as optim
        from torch.utils.data import Dataset, DataLoader
        print('Installed torch version', torch.__version__)
    except Exception as e:
        print('Could not install torch automatically. Please install PyTorch
manually and re-run the notebook.')
        raise
else:
    print('torch already available')
```

```
torch already available
```

```
# 2. Download MovieLens 100K if missing, and load ratings
import zipfile, urllib.request
if not DATA_DIR.exists() or not RATING_FILE.exists():
    print('Downloading MovieLens 100K...')
    zip_path = Path('ml-100k.zip')
    urllib.request.urlretrieve(DATA_URL, zip_path)
    with zipfile.ZipFile(zip_path, 'r') as z:
        z.extractall('.')
    zip_path.unlink()
    print('Downloaded and extracted MovieLens 100K')
else:
    print('Dataset present at', DATA_DIR)

cols = ['user_id','item_id','rating','timestamp']
ratings = pd.read_csv(RATING_FILE, sep='\t', names=cols, engine='python')
ratings = ratings.drop(columns=['timestamp'])
ratings['user'] = 'u' + ratings['user_id'].astype(str)
ratings['item'] = 'i' + ratings['item_id'].astype(str)
ratings = ratings[['user','item','rating']]
print('Loaded', len(ratings), 'ratings')
```

```
Dataset present at ml-100k
Loaded 100000 ratings
```

```
# 3. Train-test split per user
np.random.seed(SEED)

def train_test_split_per_user(df, test_frac=TEST_FRAC, seed=SEED):
    np.random.seed(seed)
    train_list=[]
    test_list=[]
    for u,g in df.groupby('user'):
        g_shuf = g.sample(frac=1.0, random_state=seed)
        n=len(g_shuf)
        if n==0: continue
        test_n = max(1, int(np.floor(n*test_frac)))
        test_list.append(g_shuf.iloc[:test_n])
        if n-test_n>0:
            train_list.append(g_shuf.iloc[test_n:])
    train_df = pd.concat(train_list).reset_index(drop=True)
    test_df = pd.concat(test_list).reset_index(drop=True)
    return train_df, test_df

train_df, test_df = train_test_split_per_user(ratings)
print('Train:', len(train_df), 'Test:', len(test_df))

Train: 80367 Test: 19633

# 4. Build user/item mappings and PyTorch Dataset
all_users = sorted(ratings['user'].unique())
all_items = sorted(ratings['item'].unique())
user_to_idx = {u:i for i,u in enumerate(all_users)}
item_to_idx = {it:i for i,it in enumerate(all_items)}
idx_to_user = {i:u for u,i in user_to_idx.items()}
idx_to_item = {i:it for it,i in item_to_idx.items()}

train_df['u_idx'] = train_df['user'].map(user_to_idx).astype(int)
train_df['i_idx'] = train_df['item'].map(item_to_idx).astype(int)

test_df['u_idx'] = test_df['user'].map(user_to_idx).astype(int)
test_df['i_idx'] = test_df['item'].map(item_to_idx).astype(int)

import torch
from torch.utils.data import Dataset, DataLoader

class RatingsDataset(Dataset):
    def __init__(self, df):
        self.u = torch.tensor(df['u_idx'].values, dtype=torch.long)
        self.i = torch.tensor(df['i_idx'].values, dtype=torch.long)
        self.r = torch.tensor(df['rating'].values, dtype=torch.float32)
    def __len__(self):
        return len(self.r)
    def __getitem__(self, idx):
        return self.u[idx], self.i[idx], self.r[idx]

train_ds = RatingsDataset(train_df)
test_ds = RatingsDataset(test_df)
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False)

n_users = len(all_users)
```

```
n_items = len(all_items)
print('n_users', n_users, 'n_items', n_items, 'train batches', len(train_loader))

n_users 943 n_items 1682 train batches 79

# 5. Define Neural Collaborative Filtering model (GMF + MLP merged)
import torch.nn as nn

class NCF(nn.Module):
    def __init__(self, n_users, n_items, emb_dim=32, mlp_layers=[64,32,16]):
        super().__init__()
        # Embeddings
        self.user_emb = nn.Embedding(n_users, emb_dim)
        self.item_emb = nn.Embedding(n_items, emb_dim)
        # MLP on concatenated embeddings
        mlp_layers_full = []
        input_dim = emb_dim * 2
        for h in mlp_layers:
            mlp_layers_full.append(nn.Linear(input_dim, h))
            mlp_layers_full.append(nn.ReLU())
            mlp_layers_full.append(nn.BatchNorm1d(h))
            input_dim = h
        self.mlp = nn.Sequential(*mlp_layers_full)
        # Final prediction layer
        self.out = nn.Linear(input_dim, 1)
        # init
        self._init_weights()
    def _init_weights(self):
        nn.init.normal_(self.user_emb.weight, std=0.01)
        nn.init.normal_(self.item_emb.weight, std=0.01)
        for m in self.mlp:
            if isinstance(m, nn.Linear):
                nn.init.xavier_uniform_(m.weight)
        nn.init.kaiming_uniform_(self.out.weight, a=1)
    def forward(self, u, i):
        u_e = self.user_emb(u)
        i_e = self.item_emb(i)
        x = torch.cat([u_e, i_e], dim=-1)
        x = self.mlp(x)
        out = self.out(x).squeeze(-1)
        return out

# create model
import torch
model = NCF(n_users, n_items, emb_dim=EMBED_DIM, mlp_layers=MLP_LAYERS).to(DEVICE)
print(model)

NCF(
  (user_emb): Embedding(943, 32)
  (item_emb): Embedding(1682, 32)
  (mlp): Sequential(
    (0): Linear(in_features=64, out_features=64, bias=True)
    (1): ReLU()
    (2): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (3): Linear(in_features=64, out_features=32, bias=True)
    (4): ReLU()
```

```
    (5): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (6): Linear(in_features=32, out_features=16, bias=True)
    (7): ReLU()
    (8): BatchNorm1d(16, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
  (out): Linear(in_features=16, out_features=1, bias=True)
)
```

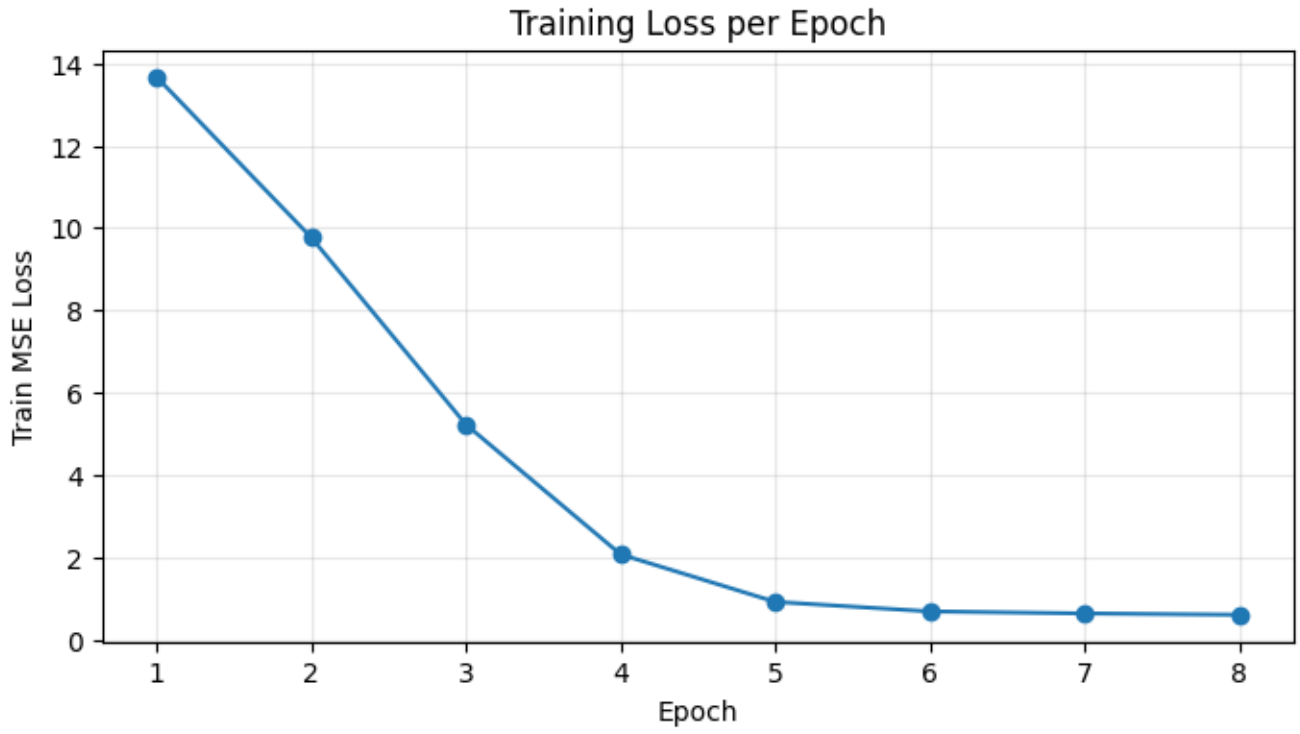
```
# 6. Training loop
import torch.optim as optim
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

train_losses = []
for epoch in range(1, EPOCHS+1):
    model.train()
    running_loss = 0.0
    n_batches = 0
    for bu, bi, br in train_loader:
        bu = bu.to(DEVICE); bi = bi.to(DEVICE); br = br.to(DEVICE)
        optimizer.zero_grad()
        preds = model(bu, bi)
        loss = criterion(preds, br)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        n_batches += 1
    avg_loss = running_loss / max(1, n_batches)
    train_losses.append(avg_loss)
    print(f'Epoch {epoch}/{EPOCHS} Train MSE loss: {avg_loss:.4f}')
```

```
Epoch 1/8 Train MSE loss: 13.6705
Epoch 2/8 Train MSE loss: 9.7702
Epoch 3/8 Train MSE loss: 5.2247
Epoch 4/8 Train MSE loss: 2.0688
Epoch 5/8 Train MSE loss: 0.9137
Epoch 6/8 Train MSE loss: 0.6824
Epoch 7/8 Train MSE loss: 0.6332
Epoch 8/8 Train MSE loss: 0.6011
```

```
# 7. Evaluate on test set (RMSE, MAE)
from sklearn.metrics import mean_squared_error, mean_absolute_error

def evaluate_model(loader, model):
    model.eval()
    y_true = []
    y_pred = []
    with torch.no_grad():
        for bu, bi, br in loader:
            bu = bu.to(DEVICE); bi = bi.to(DEVICE)
            preds = model(bu, bi).cpu().numpy()
            y_pred.extend(preds.tolist())
            y_true.extend(br.numpy().tolist())
    rmse = sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
```

```
    return {'rmse': rmse, 'mae': mae, 'n': len(y_true)}

train_eval = evaluate_model(train_loader, model)
test_eval = evaluate_model(test_loader, model)
print('Train:', train_eval)
print('Test:', test_eval)

Train: {'rmse': 0.7162034925589817, 'mae': 0.5561479402040553, 'n': 80367}
Test: {'rmse': 1.0211888144034154, 'mae': 0.8026169861776389, 'n': 19633}

# 8. Top-N evaluation (Precision@K, Recall@K) - batched scoring for speed
# Precompute user->train seen set
from collections import defaultdict
user_seen = defaultdict(set)
for _, row in train_df.iterrows():
    user_seen[row.u_idx].add(row.i_idx)

# Function to recommend top-n items for a user by scoring all items using model
@torch.no_grad()
def recommend_top_n_for_user(model, user_idx, top_n=TOPN, exclude_seen=True,
batch_size=1024):
    model.eval()
    device = next(model.parameters()).device
    u = torch.tensor([user_idx], dtype=torch.long, device=device)
    # repeat user for all items in batches
    scores = []
    item_indices = np.arange(n_items)
    for start in range(0, n_items, batch_size):
        end = min(n_items, start+batch_size)
        bi = torch.tensor(item_indices[start:end], dtype=torch.long, device=device)
        bu = u.repeat(len(bi))
        preds = model(bu, bi).cpu().numpy()
        scores.append(preds)
    scores = np.concatenate(scores)
    # apply mask for seen
    candidates = item_indices
    if exclude_seen:
        mask = np.array([idx not in user_seen[user_idx] for idx in candidates])
        candidates = candidates[mask]
        scores = scores[mask]
    top_idx_local = np.argsort(scores)[-top_n:][::-1]
    return [int(candidates[i]) for i in top_idx_local]

# Precision/Recall evaluation
def precision_recall_at_k_model(model, test_df, top_n=TOPN):
    precisions = []
    recalls = []
    users_eval = 0
    test_per_user = defaultdict(list)
    for _, row in test_df.iterrows():
        test_per_user[row.u_idx].append(row.i_idx)
    for u, true_list in test_per_user.items():
        true_set = set(true_list)
        recs = recommend_top_n_for_user(model, u, top_n=top_n)
        hit = len(set(recs) & true_set)
        precisions.append(hit / top_n)
        recalls.append(hit / len(true_set))
```

```
        users_eval += 1
    return {'precision': float(np.mean(precisions) if precisions else 0.0),
            'recall': float(np.mean(recalls) if recalls else 0.0),
            'users_eval': users_eval}

pr = precision_recall_at_k_model(model, test_df, top_n=TOPN)
print('Precision@%d = %.4f, Recall@%d = %.4f (users=%d)' % (TOPN, pr['precision'],
TOPN, pr['recall'], pr['users_eval']))

Precision@10 = 0.0245, Recall@10 = 0.0116 (users=943)

# 9. Plots
plt.figure(figsize=(8,4))
plt.plot(range(1, len(train_losses)+1), train_losses, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Train MSE Loss')
plt.title('Training Loss per Epoch')
plt.grid(alpha=0.3)
plt.show()

print('Final evaluations:')
print('Train:', train_eval)
print('Test:', test_eval)
print('Precision/Recall@%d:' % TOPN, pr)
```




```
Final evaluations:
Train: {'rmse': 0.7162034925589817, 'mae': 0.5561479402040553, 'n': 80367}
Test: {'rmse': 1.0211888144034154, 'mae': 0.8026169861776389, 'n': 19633}
Precision/Recall@10: {'precision': 0.02449628844114528, 'recall':
0.0115741311166005, 'users_eval': 943}
```

```
# 10. Save model and evaluation summaries
import pickle, json
out_dir = Path('./nn_model_based_movielens')
out_dir.mkdir(parents=True, exist_ok=True)
# Move model to cpu before saving
cpu_model = model.to('cpu')
with open(out_dir / 'ncf_model.pt', 'wb') as f:
    torch.save(cpu_model.state_dict(), f)

summary = {'train_eval': train_eval, 'test_eval': test_eval, 'precision_recall':
pr}
with open(out_dir / 'ncf_eval_summary.json', 'w') as f:
    json.dump(summary, f, indent=2)
print('Saved model and summaries to', out_dir)

# Example recommendations
for u in range(3):
    recs = recommend_top_n_for_user(model.to(DEVICE), u, top_n=5)
    rec_items = [idx_to_item[idx] for idx in recs]
    print('User', idx_to_user[u], '->', rec_items)

Saved model and summaries to nn_model_based_movielens
User u1 -> ['i1449', 'i945', 'i814', 'i488', 'i1628']
User u10 -> ['i1642', 'i272', 'i1449', 'i251', 'i1450']
User u100 -> ['i306', 'i1450', 'i647', 'i1512', 'i603']
```